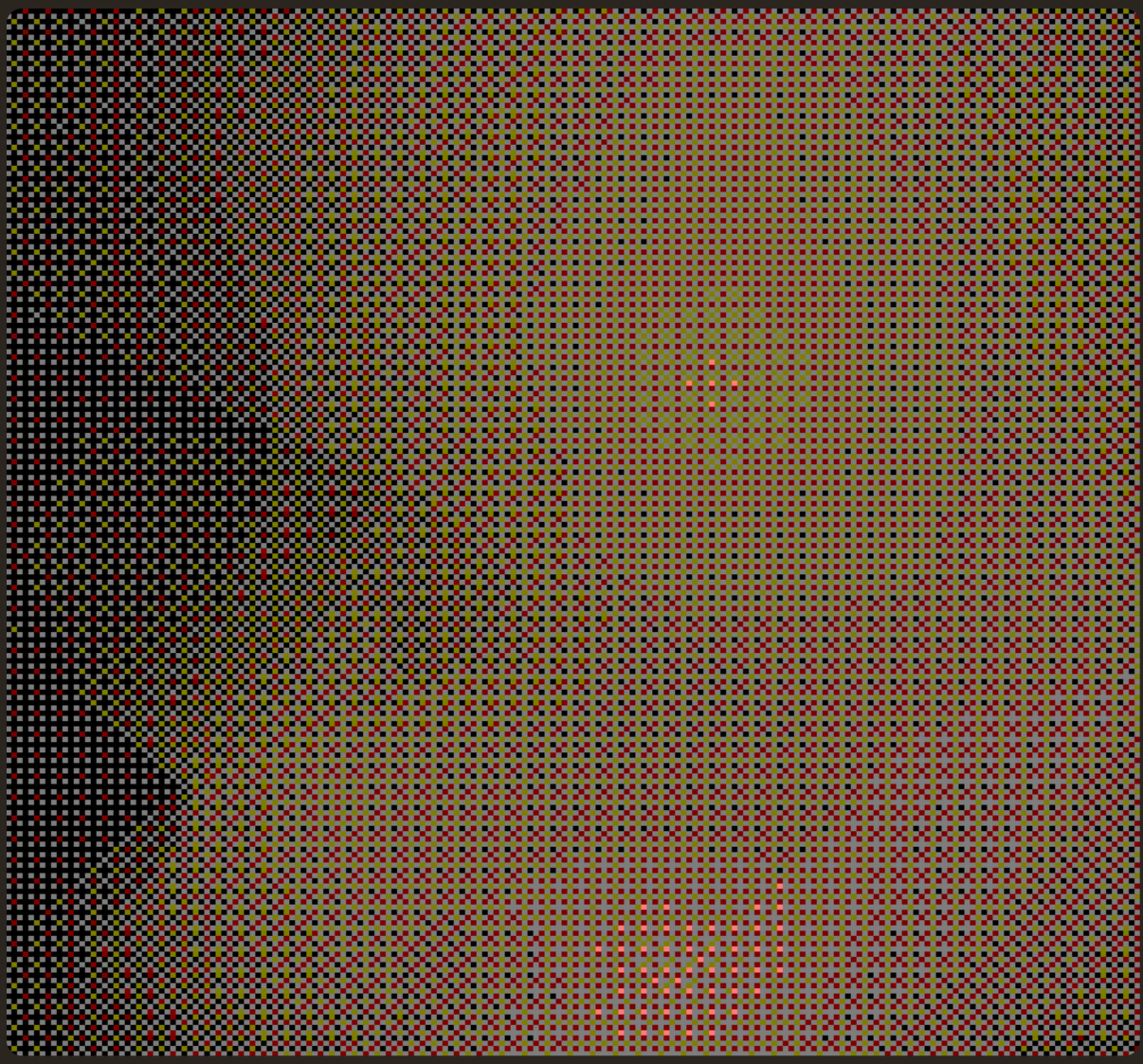
ZKS

ZKSECURITY
AUDIT REPORT

Zolana Audit

HELIUS • AUGUST 3RD, 2026



ZKSECURITY.XYZ

Contents

Zolana Audit

HELIUS / AUGUST 3RD, 2026

REPORT

01 **Introduction** →

Scope →

Recommendations →

02 **Findings** →

§01

Introduction

On August 3rd, 2026, Helius engaged zkSecurity to perform a security audit of Zolana, its private payment protocol for Solana. Over the course of three weeks, two consultants reviewed the zero-knowledge circuits and the Solana programs that make up the protocol. A number of observations and findings have been reported to the Helius team. The findings are detailed in the latter section of this report.

Scope

The scope of the audit covered the helius-labs/zolana repository, at commit `d052a2e5fc619258cca5ad901927a28e9e72fd4b`, and was divided into two parts:

- **The gnark circuits**, which enforce the correctness of shielded transactions: the spending of existing UTXOs, the creation of new ones, and the various privacy policies attached to rings.
- **The Solana shielded pool program (SPP)**, which maintain the shielded pool on-chain (commitment tree, nullifier tree, ring configurations, asset accounting) and enforce its state transitions.

Dependencies of the codebase, most notably the gnark proving library itself and the pinocchio framework, were considered out of scope and assumed to be secure, with the exception of the specific gnark behaviors discussed in this report.

In addition, the user registry program present in the same repository, and specific rings were not in scope as well.

Recommendations

In addition to the findings discussed later in the report, we have the following strategic recommendations:

Fix the findings. Two findings are of high severity and should be addressed first: Unauthorized Spending of P-256-Owned UTXOs Due to Gnark's Underconstrained Fake-GLV Implementation and Public private_tx_hash Leaks the Spent UTXO, Breaking Spend Unlinkability. Consider re-auditing the circuits once the fixes have landed, as several of the fixes touch the transaction transcript and the nullifier derivation.

Audit what was left out of scope. The hybrid encryption ring and the user registry program were not reviewed (see for example zones/squads/program/src), and both deserve their own audit: ring programs enforce their policy entirely on their own, before the CPI, and the user registry acts as the key-binding oracle for the whole system. The same applies to the wallet, as a number of the protocol's guarantees are only met if the client behaves correctly and the circuits cannot enforce that. The Faerie Gold finding is the clearest example, as avoiding crediting a user for an unspendable duplicate payment is currently a wallet responsibility.

Specify the threat model for a shared `nullifier_secret`. The design deliberately allows handing the `nullifier_secret` to a third party, like a proving server or a service that syncs and merges a user's UTXOs, and the merge circuit requires no signature as a result. The boundaries of what that delegation grants are currently implicit: the secret does not rotate, a delegate can link the spends of every UTXO whose ciphertext it was given, and it can consume inputs. We recommend specifying, per role, what a delegate is trusted with and what the intended revocation story is.

§02

Findings

Below are listed the findings found during the engagement. High severity findings can be seen as so-called "priority 0" issues that need fixing (potentially urgently). Medium severity findings are most often serious findings that have less impact (or are harder to exploit) than high-severity findings. Low severity findings are most often exploitable in contrived scenarios, if at all, but still warrant reflection. Findings marked as informational are general comments that did not fit any of the other criteria.

HIGH

#00 Unauthorized Spending of P-256-Owned UTXOs Due to Gnark's Underconstrained Fake-GLV Implementation
`prover/server/circuits/spp_transaction/custom/p256.go`

#01 Public private_tx_hash Leaks the Spent UTXO, Breaking Spend Unlinkability
*

MEDIUM

#02 Distinguishable private_tx_hash Digests Leak Real and Dummy Output Positions
`prover/server/circuits/spp_transaction/shared/outputs.go`

#03 Duplicate UTXOs Could Allow for Faerie Gold Griefing Attacks
`prover/server/circuits/spp_transaction/shared/inputs.go`

LOW

#04 Anonymous Ring Ciphertext Length Leaks the Recipient Count
`sdk-libs/transaction/src/serialization/anonymous.rs`

#05 Double Hashing Probe Sequence Can Have Short Cycles
program-libs/bloom-filter/src/lib.rs

#06 Neither Tree Account Is Authenticated by the Proof
programs/shielded-pool/src/instructions

INFORMATIONAL

#07 Choice of Bloom Filter Size and Number of Probes
program-libs/batched-merkle-tree/src/constants.rs

#08 Owner Identities Are Not Domain-Separated by Signature Algorithm
sdk-libs/keypair/src/pubkey.rs

HIGH · #00

Unauthorized Spending of P-256-Owned UTXOs Due to Gnark's Underconstrained Fake-GLV Implementation

`prover/server/circuits/spp_transaction/custom/p256.go`

Description. The `CustomRingP256Circuit` verifies a P-256 ECDSA signature inside the circuit to authorize P-256-owned UTXO inputs. The circuit uses gnark's `P256Pub.IsValid`, which delegates ECDSA signature verification to gnark's emulated gadget.

However, at the time of this engagement, the version of the gadget in use (`v0.15.0`) is affected by the soundness bug described in gnark issue #1767.

Impact. A malicious prover can forge a P-256 ECDSA signature pair (r, s) that the `CustomRingP256Circuit` accepts as valid for a known public key Q and message scalar m, without possessing the corresponding P-256 signing secret. This defeats the P-256 signing-key authorization that `CustomRingP256Circuit` enforces on P-256-owned UTXO inputs and can result in unauthorized UTXO spending.

The bypass does not, by itself, reveal an unknown UTXO opening or nullifier secret: the remaining spend constraints (owner integrity, Merkle inclusion, balance) are untouched. Observing or front-running an honest transaction exposes the public UTXO commitment, nullifier, proof, and transaction hash, but not the private opening (especially the `nullifier_secret`) needed to construct a different proof, and the honest proof remains bound to its original transaction outputs.

Practical exploitation therefore requires another source of the private witness, such as an untrusted delegated prover that receives the witness from the wallet, a compromised proving environment, or an attacker who already possesses the target UTXO's opening and `nullifier_secret`. Given the witness, the attacker can authorize an attacker-selected transaction and spend the UTXO without the owner's P-256 signing secret.

Technical Details. For an ECDSA signature (r, s) over message m and public key Q, gnark computes:

$$u_1 = ms^{-1} \pmod{n}$$

$$u_2 = rs^{-1} \pmod{n}$$

and verifies that the resulting x-coordinate equals r:

$$(x, y) = [u_1]G + [u_2]Q$$

$$r \equiv x \pmod{n}$$

To reduce circuit cost, gnark uses its optimized fake-GLV implementation for these non-native P-256 scalar multiplications. Instead of directly computing $R = [u]P$, fake-GLV obtains the following values from off-circuit hints:

- A claimed multiplication result R.
- Two allegedly short subscalars s_1 and s_2.
- A sign bit.

The circuit verifies a full scalar-field relation similar to:

$$s_1 + us_2 = 0 \pmod{n}$$

and an elliptic-curve relation:

$$[s_1]P + [s_2]R = \mathcal{O}$$

These relations establish $R = [u]P$ only when s_1 and s_2 are constrained to the expected 128-bit range.

The vulnerable implementation checks the scalar relation using the complete, potentially 256-bit, hinted values. However, the circuit consumes only the low 128 bits of each subscalar, whereas the high bits are not constrained to zero. Consequently, the circuit actually combines two different relations:

$$s_1 + u s_2 = 0 \pmod{n}$$

and the following elliptic-curve relation:

$$[\mathrm{low}_{128}(s_1)]P + [\mathrm{low}_{128}(s_2)]R = \mathcal{O}$$

For a simple example, let $H = 2^{128}$ and $u = n - H - 1$. A malicious prover can provide:

$$s_1 = H + 1, \qquad s_2 = 1, \qquad R = -P$$

The full-width scalar relation passes because:

$$(H + 1) + (n - H - 1) = n = 0 \pmod{n}$$

while the curve accumulator sees only:

$$\mathrm{low}_{128}(s_1) = 1, \qquad \mathrm{low}_{128}(s_2) = 1$$

and therefore accepts:

$$P + (-P) = \mathcal{O}$$

This makes the circuit accept the false statement $[u]P = -P$.

To construct the ECDSA forgery, the prover first chooses an arbitrary nonzero s and two attacker-selected multiplication scalars v_1 and v_2. It then computes the point T_f and defines r as follows.

$$T_f = [v_1]G + [v_2]Q$$

$$r = x(T_f), \qquad 1 \le r < n$$

Given the constructed pair (r, s), the circuit derives the two ECDSA verification scalars below.

$$u_1 = m s^{-1} \pmod{n}$$

$$u_2 = rs^{-1} \pmod{n}$$

The prover must now make the circuit treat $[v_1]G$ and $[v_2]Q$ as $[u_1]G$ and $[u_2]Q$, respectively. The same construction is applied independently to each pair (u_i, v_i), using G for the first multiplication and Q for the second.

For each desired fake scalar v_i, the prover applies LLL reduction to the two-dimensional lattice below.

$$L(v_i) = \langle (n, 0), (-v_i, 1) \rangle.$$

Every vector (x, y) in this lattice satisfies $x + v_i y \equiv 0 \pmod{n}$. LLL produces short basis vectors, after which a small-combination search yields low-half values l_1 and l_2 and a sign e satisfying the relation below.

$$l_1 + e v_i l_2 \equiv 0 \pmod{n}$$

$$0 < l_1, l_2 < 2^{128}$$

This first lattice step produces the values that the scalar-multiplication check will read from the low 128 bits, making that check behave as if it were multiplying by the attacker-selected scalar v_i. The low halves must then be embedded into full-width hint values that satisfy the separate scalar-field constraint for the circuit-derived scalar u_i.

To describe that embedding, let A and B denote the integer values placed above the low 128 bits of the full-width hint subscalars a and b, respectively. Equivalently, A and B are the high-half quotients obtained when a and b are divided by 2^{128}, while l_1 and l_2 are the corresponding remainders. The resulting base-2^{128} decompositions are shown below.

$$a = l_1 + 2^{128} A$$

$$b = l_2 + 2^{128} B$$

Because the hint values must remain in the scalar-field range $0 < a, b < n$, the high halves must satisfy the following bounds.

$$0 \le A \le \left\lfloor \frac{n-1-l_1}{2^{128}} \right\rfloor$$

$$0 \le B \le \left\lfloor \frac{n-1-l_2}{2^{128}} \right\rfloor$$

Substituting these expressions into the full-width constraint $a + eu_i b \equiv 0 \pmod{n}$ reduces the problem of choosing the high halves to the following affine congruence:

$$A + eu_i B \equiv -\left(l_1 + eu_i l_2\right)\left(2^{128}\right)^{-1} \pmod{n}.$$

A second lattice closest-vector computation finds values of A and B that satisfy both the affine congruence and these range bounds. The two computations therefore produce one pair of full-width hint values with two different meanings: their complete values satisfy the circuit's scalar equation for u_i, while their low 128-bit halves satisfy the elliptic-curve equation for v_i.

$$a + eu_i b \equiv 0 \pmod{n}$$

$$l_1 + ev_i l_2 \equiv 0 \pmod{n}$$

The prover performs this construction once for (u_1, v_1) and once for (u_2, v_2). The malicious scalar-multiplication hints then return the following points:

$$R_1 = [v_1]G$$

$$R_2 = [v_2]Q$$

Because the low-half relations make the defective scalar-multiplication checks accept R_1 and R_2 as the results of multiplication by u_1 and u_2, respectively, the circuit obtains the point below.

$$R_1 + R_2 = [v_1]G + [v_2]Q = T_f$$

Because the prover defines $r = x(T_f)$ at the start of the construction, the circuit's final x-coordinate comparison succeeds and the circuit marks (r, s) as a valid signature. The malicious prover can then produce a Groth16 proof satisfying the defective constraints without knowing the P-256 signing secret.

Recommendation. Upgrade and pin to a gnark release v0.16.0 or above containing the fix from PR #1778.

HIGH · #01

Public private_tx_hash Leaks the Spent UTXO, Breaking Spend Unlinkability

*

Description. All transactions in the circuits publish a deterministic `private_tx_hash`:

$$\text{private_tx_hash} = \text{Poseidon}(C_i, C_o, 0, E)$$

where C_i, C_o, and E denote the input UTXO commitment, output UTXO commitment, and public `external_data_hash`, respectively. The zero value represents the UTXO address commitment, which is zero for a normal UTXO transaction.

Since all of these values are publicly known, an observer can use `private_tx_hash` as a UTXO spend oracle by enumerating all candidate commitments from the state tree and comparing each result with the actual `private_tx_hash` published in the on-chain transaction event. Because the event also contains the published `nullifier`, any successful match from the enumeration reveals the exact relationship between the `nullifier` and the corresponding UTXO commitment, thereby revealing that the UTXO has been spent. Conversely, given a known UTXO commitment, anyone can scan subsequent transactions to determine whether and when the UTXO was spent.

Note that for a transaction spending k UTXOs, the attack requires only $O(n^k)$ Poseidon enumerations, where n denotes the number of candidate UTXOs for nullifier $\Rightarrow$ UTXO linking or the number of candidate transactions for UTXO $\Rightarrow$ nullifier linking, depending on the linking direction. This scaling, especially for $k = 1$, makes the attack highly practical even if the pool already contains millions of transactions.

Impact. An observer using only public on-chain values can link a `nullifier` to the exact UTXO commitment that was spent, essentially breaking the core spend unlinkability guarantee and potentially allowing transaction-graph reconstruction.

In scenarios where the UTXO owner is known, such as in the default ring or when the sender is an adversary, this could disclose balances after enough transactions, especially with the future withdrawal functionality, which involves a public amount.

More importantly, this issue is particularly severe in the case of the `CustomRingEddsaOnly` circuit. By design, the recipients are hidden while the senders are public Ed25519 signers. Therefore, if a recipient receives a UTXO in one transaction and later spends it in a second transaction, the spend linkage reveals that the public signer in the second transaction was the hidden recipient of the first transaction.

Recommendation. Ensure that no publicly available digest commits to a value that is private to the transaction but can be drawn from a publicly enumerable set. The binding should be constructed such that recomputing the digest from public data reveals nothing about which notes were spent, while still binding the transaction to its inputs, outputs, and instruction data as it does currently.

MEDIUM · #02

Distinguishable private_tx_hash Digests Leak Real and Dummy Output Positions

prover/server/circuits/spp_transaction/shared/outputs.go

Description. The transact circuits publish the UTXO output commitments, including the dummy UTXOs, to hide the output count of a single transaction. However, `ConstrainOutput` folds a zero digest for a dummy output and only folds `utxoHash` if it is a real output:

```
return api.Select(isUtxo, utxoHash, frontend.Variable(0))
```

This creates a distinguishable property in the `private_tx_hash`: the published UTXO outputs (in the `utxoHash` form) differ from those folded into the `private_tx_hash`, allowing an observer to enumerate the dummy positions in a similar fashion to the spend linkability finding.

Consider a simple transaction using a 2-input $\Rightarrow$ 3-output circuit that produces and publishes one real UTXO and two dummy UTXO outputs:

$$(C_{o,1}, C_{o,2}, C_{o,3})$$

where we assume that $C_{o,1}$ and $C_{o,2}$ are the dummy UTXO outputs.

The `HashChain` H used to calculate the UTXO output commitment chain is computed over $(0, 0, C_{o,3})$, which differs from the published UTXO outputs. Therefore:

$$\text{private_tx_hash} = \text{Poseidon}(C_i, H(0, 0, C_{o,3}), 0, E)$$

An observer can easily enumerate candidate zero masks over the output slots and find the correct positions of the dummy UTXOs by matching each candidate against the actual on-chain `private_tx_hash`. This amounts to a binary enumeration of k output positions, where k is the number of outputs ($k = 8$ for the maximum circuit shape), giving at most 2^k candidates.

Impact. An observer with knowledge of only public on-chain values can determine exactly which output slots are dummies and therefore determine the real output count of a transaction. This essentially breaks the privacy that the dummy padding is designed to provide.

Additionally, knowing which slots are real tells the observer which of the published owner tags belong to actual recipients, so the leak can narrow the candidate set for transaction-graph reconstruction.

Recommendation. We suggest folding the `utxoHash` for every output slot, both real and dummy. Since the dummy UTXO is already formed with a freely chosen blinding value, the digest cannot be enumerated without knowing the correct blinding preimage.

MEDIUM · #03

Duplicate UTXOs Could Allow for Faerie Gold Griefing Attacks

prover/server/circuits/spp_transaction/shared/inputs.go

Description. A malicious user can trick a recipient into thinking that they have received multiple payments of the same asset, while in reality the recipient is able to spend only one of them. This griefing attack is possible because the purported payments share the same UTXO commitment, which makes only one of them spendable.

This is possible because a UTXO's commitment is a function of its contents alone:

```
// prover/server/circuits/spp_transaction/shared/utxo.go
func (u UtxoCircuitFields) DefineGadget(api frontend.API) interface{} {
    ownerUtxoHash := gadgetlib.PoseidonHash(api,
[]frontend.Variable{u.Owner, u.Blinding})
    ringHash := gadgetlib.PoseidonHash(api,
[]frontend.Variable{u.RingDataHash, u.RingProgramID})
    return gadgetlib.PoseidonHash(api, []frontend.Variable{
        u.Domain,
        u.Asset,
        u.Amount,
        u.DataHash,
        ringHash,
        ownerUtxoHash,
    })
}
```

One can produce the same UTXO by using the same asset, amount, data hash, ring, recipient information (public key hash and nullifier public key), and blinding (a free variable). The on-chain UTXO tree happily stores two identical UTXO commitments.

A nullifier associated with a UTXO is derived solely from that UTXO:

```
// prover/server/circuits/spp_transaction/shared/inputs.go
func (gadget NullifierGadget) DefineGadget(api frontend.API) interface{} {
    return gadgetlib.PoseidonHash(api, []frontend.Variable{
```

```
            gadget.UtxoHash,
            gadget.Blinding,
            gadget.NullifierSecret,
        })
    }
```

Hence, two UTXOs sharing the same content produce the exact same nullifier. Consuming one of the UTXOs therefore prevents consumption of the other. This is the attack Zcash refers to as the **Faerie Gold attack** (Zcash Protocol Specification, Version v2026.7.0-113-g5d330a [NU6.2], section 8.4, page 149), in which a recipient is sent two notes, both of which verify as validly received, but only one of which can be spent.

Zcash shielded pools prevent this in different ways. They either derive a **unique** `blinding` value (called ρ in Zcash) from the position of the note being spent in the UTXO tree or produce the nullifiers within the same shielded transfer.

While the first-party wallet does not seem vulnerable because it deduplicates commitments, protecting against this griefing attack requires wallets to synchronize with the whole on-chain history and ensure that they do not notify users of duplicate payments received. This is fragile, and hence shielded pools usually protect against these kinds of attacks.

Note that the attack requires a sender who deliberately reuses a `blinding` value. Reusing this value is easier because on-chain deposits to the shielded pool publicly display the blinding value in default-ring transfers.

Recommendation. Ensure that nullifiers are unique across both the transact and deposit on-chain operations.

LOW · #04

Anonymous Ring Ciphertext Length Leaks the Recipient Count

sdk-libs/transaction/src/serialization/anonymous.rs

Description. Anonymous ring transactions use fixed input and output circuit shapes with dummy UTXOs intended to conceal the transaction's actual cardinality. However, the encrypted sender bundle `AnonymousTransferSenderPlaintext` contains `recipient_viewing_pks`, which is serialized as a variable-length vector of compressed P-256 public keys.

```
pub struct AnonymousTransferSenderPlaintext {
    #[wincode(with = "PublicKeySchema")]
    pub owner_pubkey: PublicKey,
    pub spl_asset_id: u64,
    pub spl_amount: u64,
    pub sol_amount: u64,
    pub blinding_seed: [u8; 32],
    #[wincode(with = "containers::Vec<P256PubkeySchema, FixIntLen<u8>>")]
    pub recipient_viewing_pks: Vec<P256Pubkey>,
    pub spl_data: Data,
    pub sol_data: Data,
}
```

Because the encryption scheme is AES-CTR, which is length-preserving, the published ciphertext grows by exactly 33 bytes for every real recipient. An observer can therefore recover the number of recipients from the on-chain ciphertext length without possessing a viewing key or decrypting the ciphertext.

Note that `spl_data` and `sol_data` are also variable-length fields, which in principle may add noise that makes the recipient count harder to read off directly. However, in practice the likelihood of this defeating the attack is low because a normal transfer attaches no application data by default, so each field serializes to a single byte denoting an empty vector.

Impact. Any passive on-chain observer can determine the exact number of recipients in an anonymous-ring transaction, even though the circuit's fixed output shape and dummy UTXOs are intended to hide this information.

This reduces the effective anonymity set and provides indirect metadata that can be combined with other related transaction data to narrow candidate recipients. The variable-length `spl_data` and `sol_data` fields lower the likelihood of a fully precise count in some scenarios, but they do not prevent the leak and must not be relied upon as a mitigation.

Recommendation. We suggest using a fixed-length encrypted representation with padding, so the ciphertext size remains constant and does not reveal the actual number of recipients.

LOW · #05

Double Hashing Probe Sequence Can Have Short Cycles

program-libs/bloom-filter/src/lib.rs

Description. The bloom filter can have a short cycle in its probe sequence, which reduces the number of distinct bits set on insertion and tested on lookup. The Bloom filter derives its probe positions by double hashing. A single Keccak digest of the value is split into two 64-bit halves, and the probe sequence walks $h_1 + i \cdot h_2 \bmod m$:

```
fn probes(value: &[u8; 32]) -> (u64, u64) {
    let hash = solana_nostd_keccak::hash(value);
    let h1 = u64::from_le_bytes(hash[0..8].try_into().unwrap());
    let h2 = u64::from_le_bytes(hash[16..24].try_into().unwrap());
    (h1, h2)
}
```

```
let mut probe = h1;
for _ in 0..NUM_ITERS {
    let probe_index = (probe % num_bits) as usize;
    probe = probe.wrapping_add(h2);
    // ...
}
```

The stride h_2 is taken directly from the digest and is not constrained in any way. When h_2 shares a factor with the table size m, the sequence does not visit k distinct positions but cycles within a proper subset of the table. The table size here is $m = 4,603,072 = 2^6 \cdot 71,923$, so an even h_2 already shares the factor 2^6 and confines every probe of that element to a single congruence class. In the degenerate case $h_2 \equiv 0 \pmod{m}$, all k probes collapse onto one bit.

Impact. For an element whose stride is degenerate, fewer than k distinct bits are set on insertion and fewer than k distinct bits are tested on lookup, so that element is checked against a weaker filter than the parameters assume. This raises the effective false positive probability above the 10^{-12} target the filter was sized for. With $k = 10$ the effect is small, since a short cycle only

costs the probes that repeat, and a false positive in the nullifier queue is a spurious rejection rather than an accepted double spend. We therefore rate this low.

Recommendation. Force the stride to be odd, so that it shares no power-of-two factor with the table size:

```
let h2 = u64::from_le_bytes(hash[16..24].try_into().unwrap()) | 1;
```

This costs one instruction and removes both the even-stride and the zero-stride cases. Note that it does not make the stride coprime with the odd part of m, so if the table size is later chosen with small odd factors, the stride should be reduced modulo m and forced into the coprime range instead.

Client Response. Fixed in commit `b7cf1f07`, which forces the stride odd as recommended and documents the reasoning.

LOW · #06

Neither Tree Account Is Authenticated by the Proof

programs/shielded-pool/src/instructions

Description. Tree accounts are used to create shielded pools in the SPP. Each shielded pool conceptually consists of a Merkle tree of UTXO commitments and a Merkle tree of nullifiers.

SPP transact and merge transactions support multiple tree accounts:

- They consume from an `input_tree` account, pointing to some of its roots and appending nullifiers to its nullifier tree.
- They store new UTXO commitments in an `output_tree`.

The choice of input and output trees is authenticated not by the proof but by the transaction signatures. In some cases where transaction signatures are not needed (generally with P-256 signatures or merge instructions), a malicious relayer can swap input trees and output trees. Effectively, this means that a malicious relayer can swap input trees as long as the root matches (which is most likely a far-fetched edge case) and swap output trees to move newly created UTXOs to the wrong shielded pool.

The following table more clearly illustrates which scenarios lack authentication.

RAIL	OWNER AUTHORIZATION COVERS	TREES AUTHENTICATED
`transact` / `ring_transact`, EdDSA	the whole Solana message, including the account keys list	yes, incidentally
`ring_transact`, P-256	`sha256(PrivateTxHash(input hashes, output hashes, external_data_hash))`	no

RAIL	OWNER AUTHORIZATION COVERS	TREES AUTHENTICATED
`ring_authority_transact`	nothing; the instruction requires no signatures (`transact/account.rs:196–200`)	no
`merge` / `merge_ring`	nothing; the instruction is permissionless and reads identity from the registry PDA	no

An EdDSA owner is protected only as a side effect of how they sign. Their key must be a Solana signer account because the circuit enforces that the owner of any UTXO being spent is among the Solana transaction signers.

The P-256 circuit, on the other hand, cannot cover the whole transaction because that would be recursive. For this reason, a hash is produced to cover as much as possible, including which notes are spent and created, the expiry, the settlement amounts and their recipient accounts, and the ring and data hashes (see the Intention Authorization section of the overview).

```
// programs/shielded-pool/src/instructions/merge/verify.rs:85–115
[ chain(nullifiers), output_utxo_hash, chain(utxo_tree_roots),
  chain(nullifier_tree_roots), private_tx_hash, external_data_hash,
  allow_dummy_inputs ]  ++  [ user_signing_pk_hash ]
```

Merge instructions are especially affected because they are permissionless by design. As a result, any observer can lift a broadcast `merge_transact` from the mempool, resubmit it verbatim with their own payer and any valid tree as `output_tree`, and land it first.

P-256-based instructions, as well as authority instructions, can be submitted by relayers who get to pick both trees themselves.

Recommendation. Add both tree addresses to the external data hash in transact proofs. For merges, let users specify the trees on which they want to allow merges.

INFORMATIONAL · #07

Choice of Bloom Filter Size and Number of Probes

program-libs/batched-merkle-tree/src/constants.rs

Description. The nullifier queue uses a Bloom filter parameterized by the following constants:

```
pub const ADDRESS_BLOOM_FILTER_CAPACITY: u64 = 4_603_072;
pub const ADDRESS_BLOOM_FILTER_NUM_HASHES: u64 = 10;
```

That is $m = 4,603,072$ bits of state (about 562 KiB) and $k = 10$ probes per element, sized for $n = 30,000$ elements per filter. The design goal is to keep the probability of a false positive very small over the whole lifetime of the tree, which at height 40 holds roughly 1.1 trillion leaves. With these values the false positive probability is

$$p = \left(1 - e^{-kn/m}\right)^k \approx 10^{-12}$$

The parameters therefore sit at a deliberate point on a tradeoff between compute, which grows with the number of hash functions k, and on-chain state, which grows with m.

We note that the chosen point is not the only one that meets the 10^{-12} target, and that the two directions of the tradeoff are available at quite different exchange rates:

- Holding m fixed and spending more compute, the optimal probe count for this m/n ratio is $k = \lceil (m/n) \ln 2 \rceil = 106$, which would lower the false positive probability to roughly $2^{-106} \approx 10^{-32}$.
- Holding the false positive probability fixed at 10^{-12} and spending more compute, the state can be reduced by a factor of about 2.7. Concretely, $m = 215,664 \cdot 8 = 1,725,312$ bits (about 211 KiB) with $k = 40$ yields the same 10^{-12}.

Recommendation. Consider benchmarking the $k = 40$ configuration against the current one. If the additional hashing fits within the compute budget of the batch update instruction, it buys back roughly two thirds of the account space at an unchanged false positive probability.

INFORMATIONAL · #08

Owner Identities Are Not Domain-Separated by Signature Algorithm

sdk-libs/keypair/src/pubkey.rs

Description. A UTXO commits to its owner as

`owner = Poseidon(pk_field, nullifier_pk)`
where `pk_field` is the hash of a bare 32-byte string whose meaning depends on which signature algorithm the owner uses: Ed25519 or ECDSA with the P-256 curve.

Nothing prevents an owner associated with a P-256 key from *pretending* that their public key is in fact an Ed25519 key (and vice versa).

While we could find no attack that turns a public key of one algorithm into a weak public key of the other algorithm, the omission of an algorithm type in the UTXO structure is surprising. An attack found against one of the algorithms or curves could weaken the accounts of users who have chosen not to use that algorithm or curve.

Furthermore, the addition of new algorithms in the future could create new problems.

For these reasons, we recommend adding a tag to the owner field to differentiate the two algorithms and enforce the user's choice of signature algorithm at a higher level.